# Chapter 21 - The Paula DMA Engine

Paula is the Amiga's four-channel sample DMA chip. It reads signed-8-bit samples from main memory and streams them into the mixer at a pitch chosen by a per-channel **period** value. Unlike MOD (Chapter 19) which embeds and decodes a whole tracker file, the Paula engine is a low-level register interface: the program provides the sample bytes, the length, the period, and the volume, then enables DMA. The chip handles the rest.

This is the engine of choice for ProTracker-style mixers written in CPU code, for streaming digitised audio (speech, custom samples), and for any program that wants direct DMA-driven sample playback without the overhead of a tracker engine.

## 21.1 What Paula can show

| Item | Value |
|---|---|
| Channels | `4` |
| Sample format | Signed 8-bit |
| Sample words | 2 bytes per word (length is in words) |
| Period | Paula PAL clock / period = sample rate |
| Volume | `0`–`64` per channel |
| Interrupts | One per channel on buffer completion |
| Clock | `3,546,895` Hz (Paula PAL) |

## 21.2 The register block

Paula lives in a 16-byte block per channel followed by three global registers, all at `0xF2260`–`0xF22AF`.

### 21.2.1 Per-channel registers

Each channel is 16 bytes; channel `n` (`n` = `0`–`3`) starts at `0xF2260 + n*16`.

| Offset | Name | Purpose |
|---|---|---|
| `0x00` | `PTR` | Sample data address (32-bit). |
| `0x04` | `LEN` | Length in **words** (1 word = 2 bytes). |
| `0x08` | `PER` | Period (output rate = `3,546,895 / PER`). |
| `0x0C` | `VOL` | Volume, `0`–`64`. Values above `64` clamp to `64`. |

The channel base addresses are:

| Channel | Base | End |
|---|---|---|
| `0` | `0xF2260` | `0xF226F` |
| `1` | `0xF2270` | `0xF227F` |
| `2` | `0xF2280` | `0xF228F` |

| Channel | Base | End |
|---|---|---|
| 3 | `0xF2290` | `0xF229F` |

### 21.2.2 Global registers

| Address | Name | Purpose |
|---|---|---|
| `0xF22A0` | `AROS_AUD_DMACON` | DMA control. Set/clear bits select operation. |
| `0xF22A4` | `AROS_AUD_STATUS` | Per-channel completion flags (read; write `1` to clear). |
| `0xF22A8` | `AROS_AUD_INTENA` | Per-channel interrupt enable. |

`DMACON` encoding:

| Bit | Meaning |
|---|---|
| 15 | `1` = set the named bits; `0` = clear them. |
| 0–3 | Channel mask (`1` = channel 0, `2` = channel 1, `4` = channel 2, `8` = channel 3). |

To enable a channel, write `(0x8000 | mask)` to `DMACON`. To disable one, write `(mask)` with bit `15` clear.

`STATUS` bits `0`–`3` are set by the chip when a channel's sample buffer is exhausted. The CPU clears them by writing the same bit back; a write of `0x0F` clears every flag.

`INTENA` uses the same set-or-clear encoding as `DMACON`. Bit `15` chooses set (`1`) or clear (`0`); bits `0`–`3` are the channel mask. When `INTENA`'s bit for a channel is set and `STATUS`'s matching bit is raised, Paula asserts an interrupt on the M68K coprocessor (auto-vector level `3`).

## 21.3 Programming model

The shortest path to play one buffer on channel `0`:

1. Place signed-8-bit sample data in memory.
2. Write the data address to channel 0's `PTR`.
3. Write the length in **words** (bytes / 2) to `LEN`.
4. Write the period to `PER`. Period = `3,546,895 / desired_rate`.
5. Write the volume (`0`–`64`) to `VOL`.
6. Write `0x8001` to `DMACON` (set, channel 0).

The chip reads samples at the chosen rate, sends them through the mixer, and on the last sample raises `STATUS` bit `0`. If `INTENA` bit `0` was also set, an interrupt is raised.

### 21.3.1 Double-buffering

After a channel starts, writing a new `PTR`/`LEN`/`PER`/`VOL` during playback stages those values as the **next** buffer. When the current buffer finishes, the chip switches to the staged buffer without a gap. This is the standard Amiga pattern for seamless streaming: the CPU prepares buffer N+1 while buffer N is playing.

## 21.4 Period and pitch

The Paula clock is `3,546,895` Hz. The output sample rate for period `PER` is

```
rate = 3546895 / PER
```

A few useful periods:

| Period | Rate (approx.) | Use |
| --- | --- | --- |
| 124 | 28,604 Hz | High-quality sample. |
| 253 | 14,019 Hz | ProTracker C-1 (PAL). |
| 508 | 6,983 Hz | Lower-rate sample. |
| 1015 | 3,494 Hz | Speech, telephone-quality. |

The minimum useful period is around `124`; values lower than that exceed the chip's ability to fetch samples at the right rate.

## 21.5 Volume

`VOL` is a 7-bit value in the range `0–64`. `0` mutes the channel; `64` is full volume. Writing more than `64` clamps to `64`. Volume changes apply at the start of the next sample.

## 21.6 BASIC keywords

There is no dedicated `PAULA` keyword. The chip is programmed by `POKE` from BASIC, or by direct stores from machine language. For higher-level playback, use the MOD player (Chapter 19), which builds its mixer on top of Paula-style sample logic; or load a tracker module through `SOUND PLAY` (Chapter 22).

A BASIC fragment that plays a sample at the standard ProTracker C-1 rate:

```
10 POKE &H000F0800, 1                : REM AUDIO_CTRL = on
20 BLOAD "sample.raw", &H200000
30 POKE &H000F2260, &H00200000        : REM ch 0 PTR
40 POKE &H000F2264, 8000              : REM 16000 bytes → 8000 words
50 POKE &H000F2268, 253               : REM PER ≈ 14 kHz
60 POKE &H000F226C, 64                : REM full volume
70 POKE &H000F22A0, &H8001            : REM DMACON: set, channel 0
```

To stop the channel, write `0x0001` to `DMACON` (clear, channel 0). To clear the completion flag afterwards, write `1` to `STATUS`.

## 21.7 Putting it together

Paula sits below every Amiga-style mixer in this machine. Use it directly when you want bit-exact ProTracker timing or when you need to stream large amounts of digitised audio that cannot live in a fixed-size SFX channel. The completion interrupt makes seamless looping (and double-buffering for longer-than-memory streams) straightforward.

The next chapter pulls every audio engine together and shows how to drive them from BASIC and from each of the six CPUs.